

📄 WEB APP IMPLEMENTATION PLAN · V2.0



KhataFlow

Web App Implementation Plan

Next-Generation Digital Credit Ledger for
Kirana Stores, Dairy Shops & Local Retailers

5

DEV PHASES

6

WEEKS BUILD

8

LANGUAGES

∞

AUDIT TRAIL

Prepared · July 2026 | KhataFlow Team | Confidential

NAVIGATION

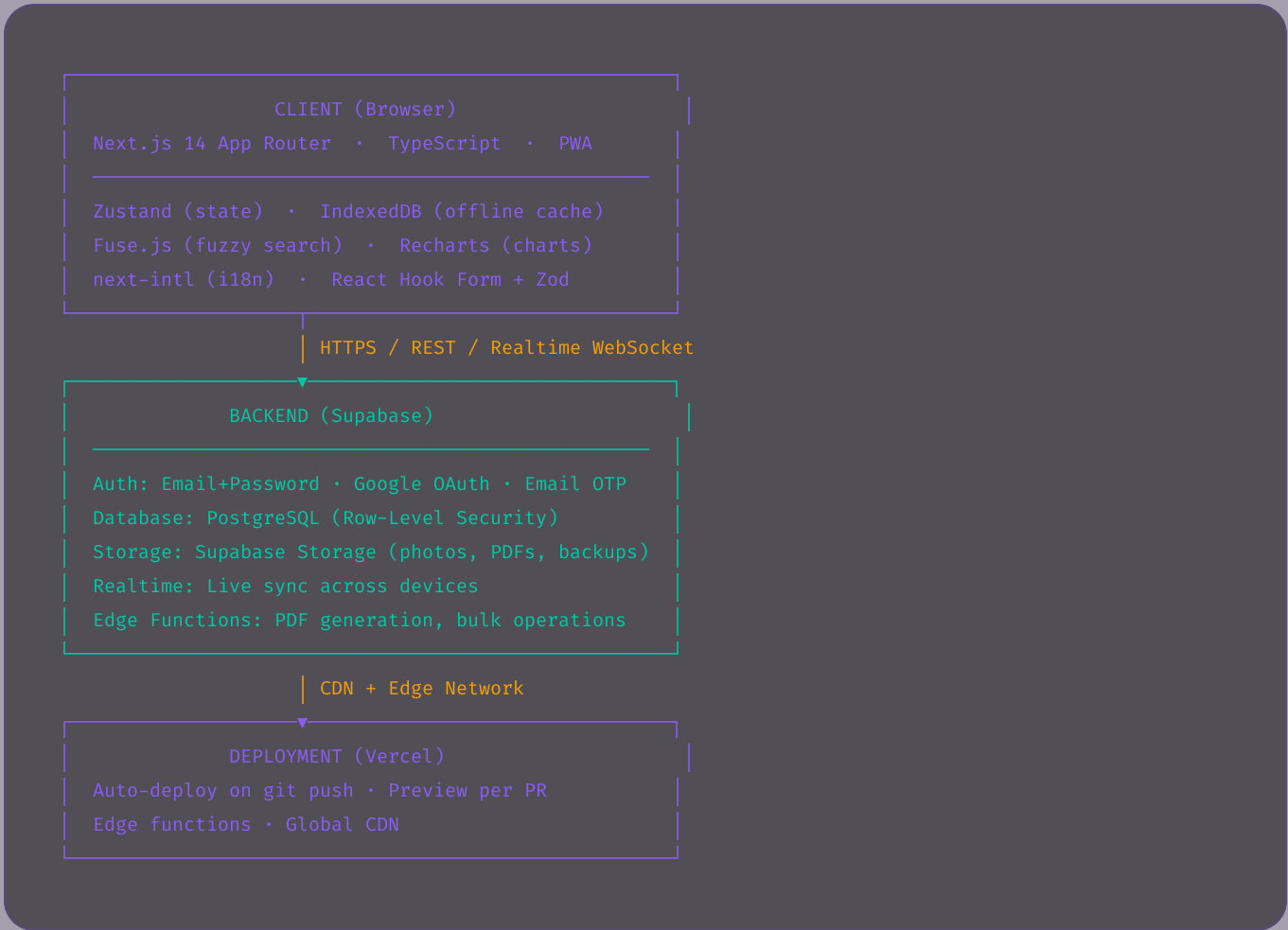
Table of Contents

- 01 Architecture Overview
- 02 Tech Stack Decisions
- 03 Language Support (i18n)
- 04 Database Schema
- 05 Project File Structure
- 06 Phase 1 — Foundation & Auth
- 07 Phase 2 — Customer Management
- 08 Phase 3 — Reports & Analytics
- 09 Phase 4 — Settings & i18n
- 10 Phase 5 — Power Features
- 11 Deployment Plan
- 12 Verification Plan

01 · ARCHITECTURE

System Architecture

A full-stack web application with offline-first capabilities, cloud sync, and real-time multi-device support.



Key Design Decisions



02 · TECHNOLOGY

Tech Stack

Every technology chosen for a specific reason — no over-engineering, no unnecessary complexity.

LAYER	TECHNOLOGY	WHY THIS CHOICE
 Framework	Next.js 14 + TypeScript	SSR, App Router, PWA support, API routes, best DX in 2025
 Styling	Vanilla CSS + CSS Variables	Full control, zero dependency, perfectly matches the design document palette
 Fonts	Inter + Poppins	Matches the KhataFlow design document exactly
 Auth	Supabase Auth	Email+password, Google OAuth, OTP, email verification — all built-in, zero config
 Database	Supabase PostgreSQL	Cloud-hosted Postgres with Row Level Security per user/shop
 File Storage	Supabase Storage	Customer photos, PDF exports, JSON backups — CDN delivered
 Offline Cache	IndexedDB via idb	Instant reads, works 100% offline, syncs to cloud on reconnect
 State	Zustand	Lightweight, TypeScript-friendly, no boilerplate, matches doc recommendation
 Server State	TanStack Query	Caching, background sync, optimistic updates, loading states
 Search	Fuse.js	Fuzzy matching across name/phone/room ID with configurable thresholds

LAYER	TECHNOLOGY	WHY THIS CHOICE
 Charts	Recharts	React-first, responsive, dark-mode friendly, D3 power
 18n	next-intl	Best-in-class Next.js internationalization with URL-based locale routing
 Forms	React Hook Form + Zod	Fast, type-safe forms with schema validation
 WhatsApp	wa.me deep link	Opens WhatsApp with pre-filled message — NO API, NO cost, instant
 Deployment	Vercel	Zero-config Next.js hosting, edge network, auto preview deployments on each PR

 WhatsApp Integration (No API Required)

When the shopkeeper clicks "Send Reminder", a modal opens with a pre-filled message template. The user can edit the message, then clicking "Send via WhatsApp" opens: `https://wa.me/91{phone}?text={encodedMessage}` — which redirects directly to WhatsApp with the message ready to send. No WhatsApp Business API, no approval, no cost.

— 03 · LOCALIZATION

Language Support

Built with next-intl — all UI strings are extracted into JSON locale files. Users choose their language in Settings; it's saved to their shop profile and persists across devices.

How Locale Routing Works

Next-intl adds a locale prefix to all URLs: `/en/customers` , `/hi/customers` . Switching language in Settings triggers a locale redirect. WhatsApp message templates are available in all installed languages.

04 · DATABASE

Database Schema

Supabase PostgreSQL with Row Level Security. Every table is scoped to a shop_id. Users can only read/write their own shop's data — enforced at the database level, not just the application level.

shops

COLUMN	TYPE	CONSTRAINT	DESCRIPTION
id	UUID	PRIMARY KEY	Unique shop identifier
owner_id	UUID	FK → auth.users.id	Account owner (from Supabase Auth)
name	VARCHAR(100)	NOT NULL	Shop name
address	TEXT	NULLABLE	Physical address
phone	VARCHAR(15)	NULLABLE	Shop contact number
gst_number	VARCHAR(15)	NULLABLE	GST registration number
plan	ENUM	DEFAULT 'FREE'	'FREE', 'PRO', 'BUSINESS'
settings	JSONB	NOT NULL	{ language, dark_mode, pin_enabled, accent_color }
created_at	TIMESTAMP TZ	NOT NULL	Shop registration date

customers

COLUMN	TYPE	CONSTRAINT	DESCRIPTION
id	UUID	PRIMARY KEY	Unique customer ID (UUID v4)
shop_id	UUID	FK → shops.id CASCADE	Which shop this customer belongs to

COLUMN	TYPE	CONSTRAINT	DESCRIPTION
name	VARCHAR(100)	NOT NULL	Customer full name
phone	VARCHAR(15)	UNIQUE, NULLABLE	Mobile number (with country code)
room_id	VARCHAR(20)	NULLABLE	Room/Flat/Hostel ID
photo_url	TEXT	NULLABLE	Supabase Storage path for customer photo
credit_limit	DECIMAL(10,2)	DEFAULT 0	Max credit allowed (0 = no limit)
advance_balance	DECIMAL(10,2)	DEFAULT 0	Overpayment wallet balance
outstanding_due	DECIMAL(10,2)	DEFAULT 0	Cached current outstanding (recalculated on each transaction)
notes	TEXT	NULLABLE	Internal shop notes about customer
tags	JSONB	DEFAULT '[]'	Custom tags array e.g. ["regular", "vip"]
family_id	UUID	FK → customers.id, NULLABLE	Head-of-family for linked accounts
is_active	BOOLEAN	DEFAULT TRUE	Soft-delete flag
created_at	TIMESTAMPTZ	NOT NULL	Account creation timestamp
updated_at	TIMESTAMPTZ	NOT NULL	Last modification (auto-updated by trigger)

 transactions

COLUMN	TYPE	CONSTRAINT	DESCRIPTION
id	UUID	PRIMARY KEY	Unique transaction ID
customer_id	UUID	FK → customers.id	Customer involved

COLUMN	TYPE	CONSTRAINT	DESCRIPTION
shop_id	UUID	FK → shops.id	Shop this transaction belongs to
type	ENUM	NOT NULL	'CREDIT', 'PAYMENT', 'ADVANCE', 'WRITEOFF', 'ADJUSTMENT', 'NOTE'
amount	DECIMAL(10,2)	NOT NULL	Transaction amount in INR
balance_before	DECIMAL(10,2)	NOT NULL	Outstanding balance before this transaction
balance_after	DECIMAL(10,2)	NOT NULL	Outstanding balance after this transaction
items	JSONB	NULLABLE	Array of { name, qty, price } for item-wise mode
note	TEXT	NULLABLE	Optional note or reason
recorded_by	UUID	FK → auth.users.id	Staff member who recorded this transaction
transaction_at	TIMESTAMPTZ	NOT NULL	Actual time of transaction (adjustable for offline entries)
is_disputed	BOOLEAN	DEFAULT FALSE	Dispute flag – shows ⚠ in timeline
dispute_note	TEXT	NULLABLE	Reason for dispute

 staff & reminder_log

COLUMN	TYPE	CONSTRAINT	DESCRIPTION
staff table			
id	UUID	PRIMARY KEY	Staff member record
shop_id	UUID	FK → shops.id CASCADE	Which shop
user_id	UUID	FK → auth.users.id	Linked Supabase Auth user
role	ENUM	NOT NULL	'OWNER', 'MANAGER', 'STAFF'
reminder_log table			
customer_id	UUID	FK → customers.id	Who was reminded

COLUMN	TYPE	CONSTRAINT	DESCRIPTION
amount_due	DECIMAL(10,2)		Amount due at time of reminder
message	TEXT		Exact message sent
sent_at	TIMESTAMPTZ	DEFAULT now()	When reminder was sent

— 05 • STRUCTURE

Project File Structure

Organized by feature and responsibility. Every file has exactly one job.

```
khataflow/ |─ app/ | |─ [locale]/ # i18n routing (next-intl) | |─ (auth)/ | | |
|─ login/page.tsx # Login – Email + Google OAuth | | |─ register/page.tsx # Sign up
+ email verification | | |─ verify/page.tsx # "Check your inbox" screen | | |─
onboarding/page.tsx # First-time shop setup | | |─ (app)/ # Protected app routes (auth
guard) | | |─ layout.tsx # App shell – sidebar or bottom nav | | |─ page.tsx #
Home Dashboard | | |─ customers/ | | | |─ page.tsx # Customer list + fuzzy search
| | | |─ [id]/page.tsx # Customer profile + timeline | | |─ reports/page.tsx #
Analytics + charts + export | | |─ settings/page.tsx # Shop profile, security,
language | | |─ layout.tsx # Root locale layout | |─ api/ | | |─
auth/callback/route.ts # Google OAuth + email magic link | | |─ export/pdf/route.ts #
PDF generation (Phase 2+) | | |─ globals.css # All CSS tokens + design system | |─
components/ | |─ auth/ LoginForm • RegisterForm • GoogleSignInButton | |─ layout/
Sidebar • BottomNav • Header | |─ customers/ CustomerSearch • CustomerCard •
CustomerAvatar • CustomerBadge | |─ modals/ AddCustomer • AddCredit • RecordPayment •
SendReminder • WriteOff • Dispute | |─ transactions/ Timeline • Entry • Filters | |─
charts/ CreditPaymentBar • OutstandingTrend • TopDebtorsPie • RecoveryGauge | |─
dashboard/ SummaryCard • OverdueAlert • RecentFeed | |─ ui/ Button • Input • Modal •
Badge • Card • Spinner • Toast | |─ lib/ | |─ supabase/ client.ts • server.ts •
middleware.ts | |─ db/ idb.ts (IndexedDB) • sync.ts (cloud sync) • schema.ts
(TypeScript types) | |─ store/ customerStore • transactionStore • shopStore • uiStore
(Zustand) | |─ finance.ts # Core business logic (credit/payment/wallet) | |─ fuzzy.ts
# Fuse.js search wrapper | |─ whatsapp.ts # wa.me URL builder + message templates |
|─ export.ts # CSV, JSON backup, print helpers | |─ utils.ts # Misc helpers | |─
messages/ en.json • hi.json • mr.json • ta.json • te.json • bn.json • gu.json • pa.json
| |─ middleware.ts # Auth guard + locale redirect | |─ i18n.ts # next-intl config | |─
supabase/ | |─ migrations/ 001_initial_schema.sql • 002_rls_policies.sql | |─ seed.sql #
Demo data for development
```

06 · PHASE 1

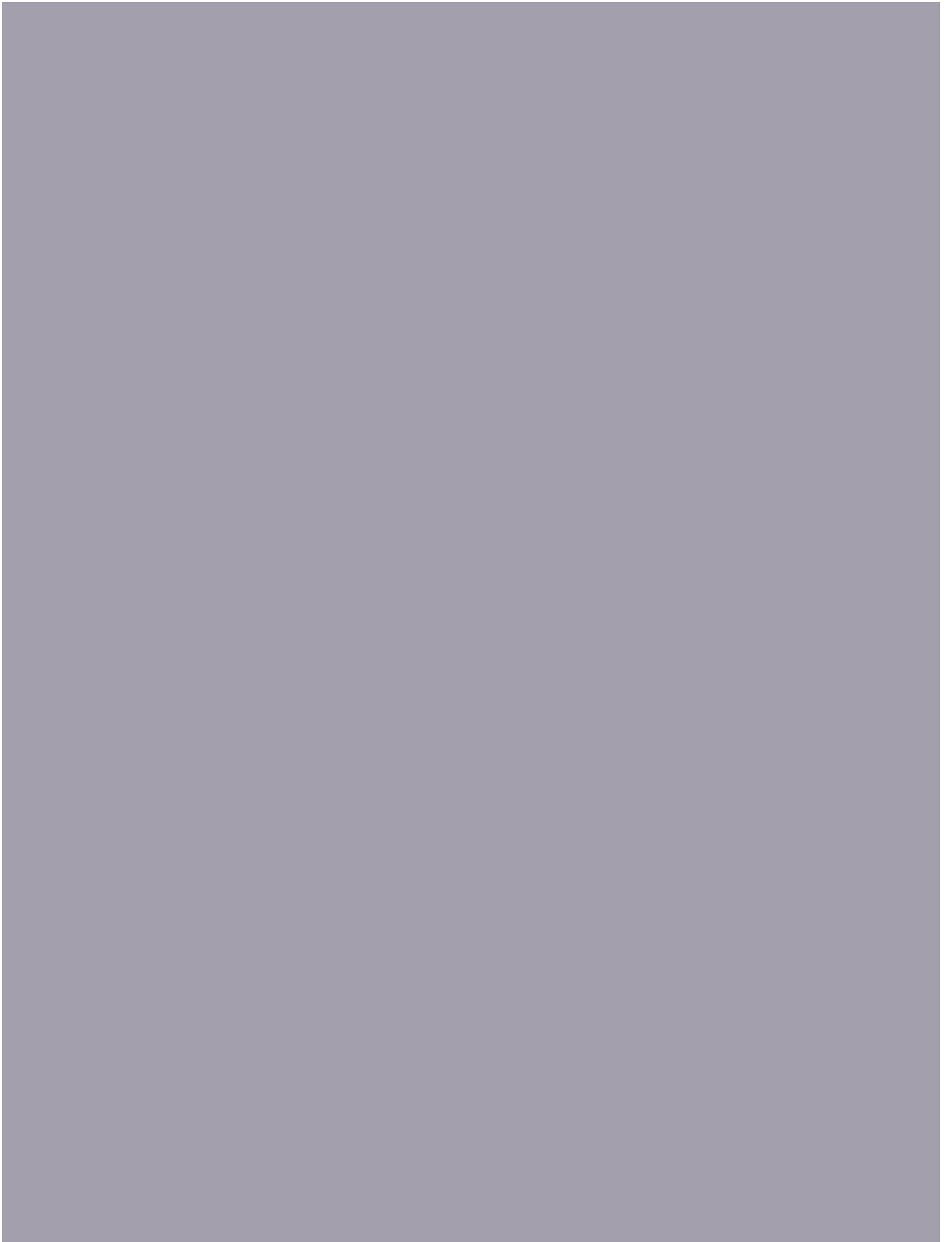
Phase 1 — Foundation & Auth

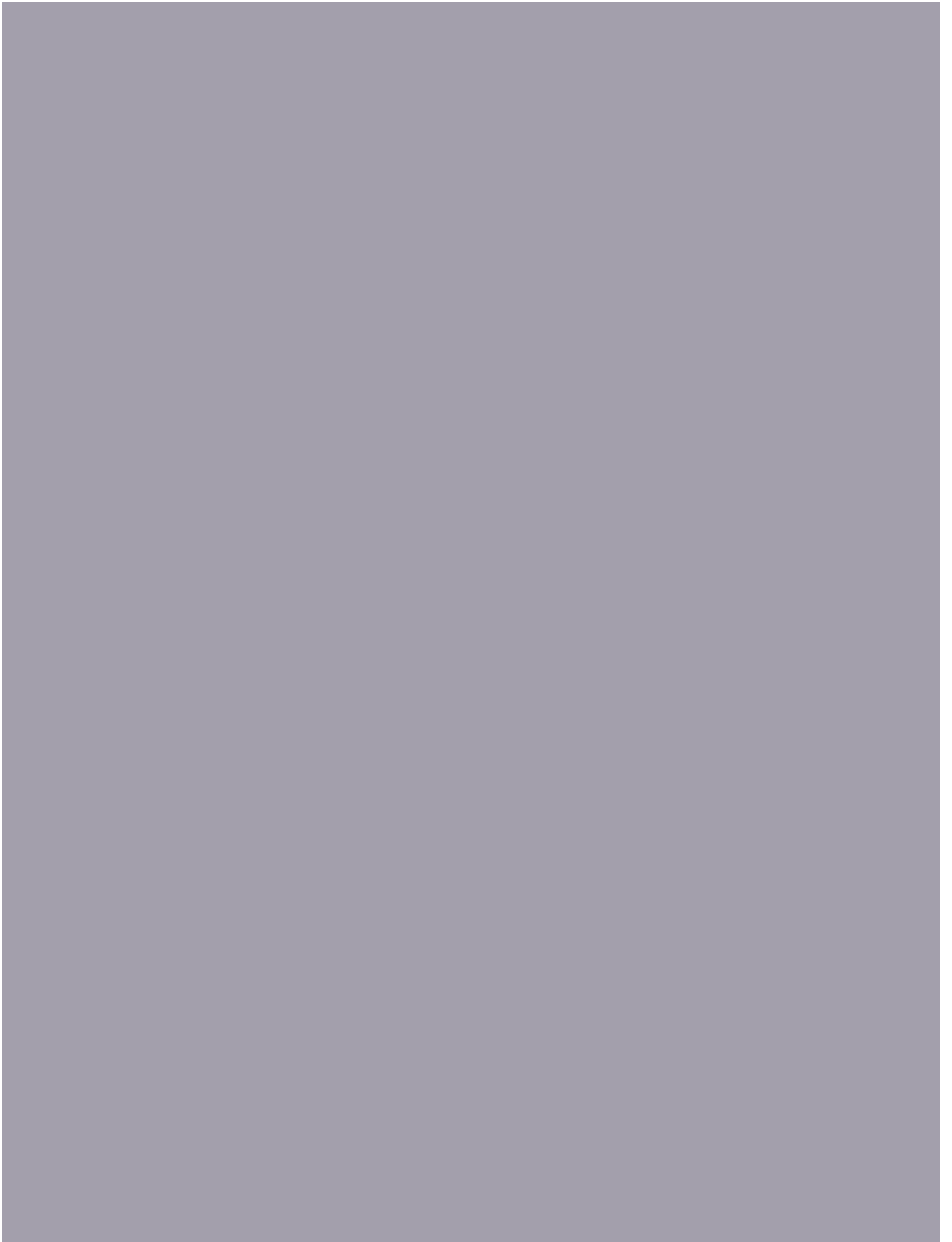
Weeks 1–2. Goal: A running app with beautiful UI, Google + email auth, email verification, and a working shop setup flow. No data yet — just the skeleton and security foundation.

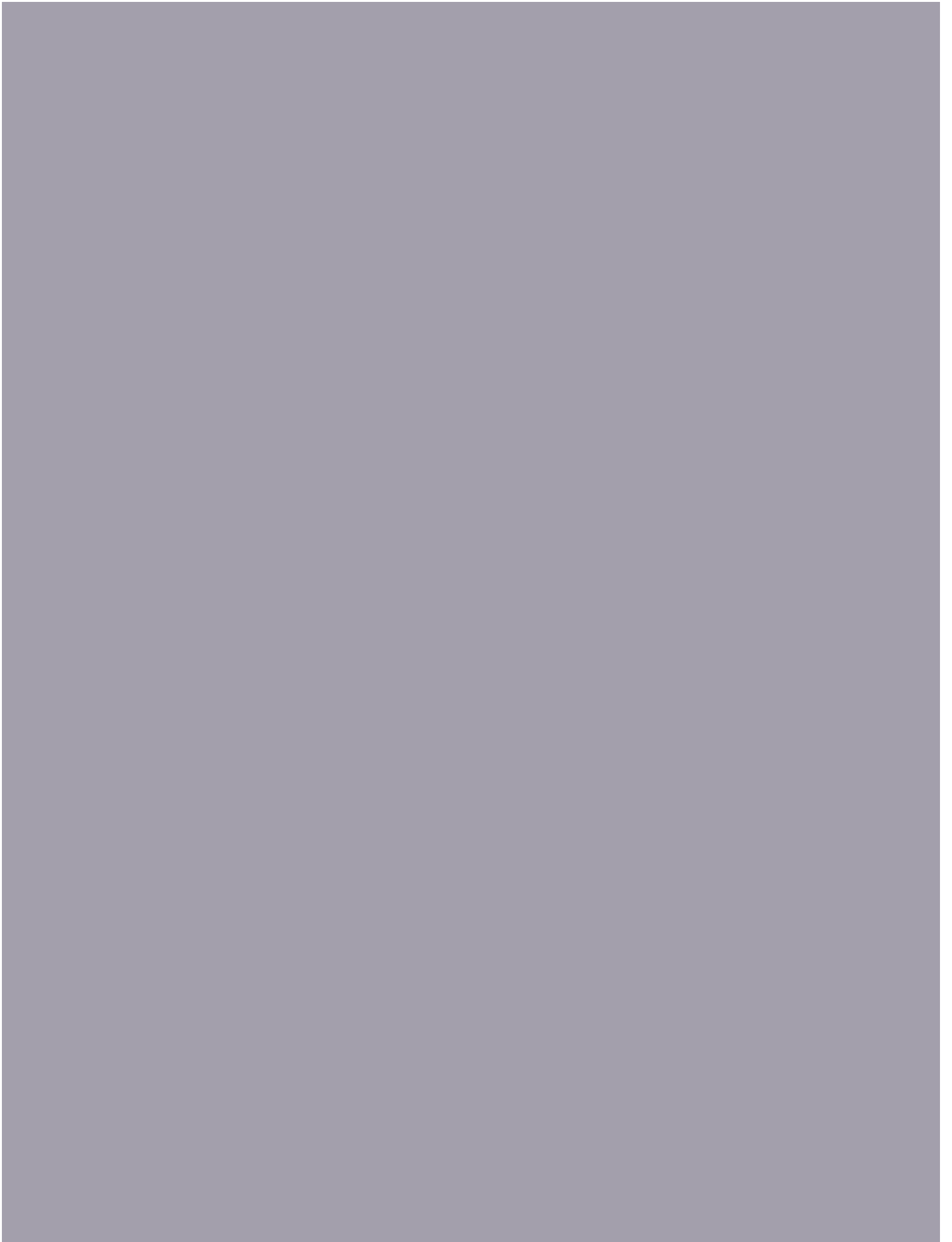
07 · PHASE 2

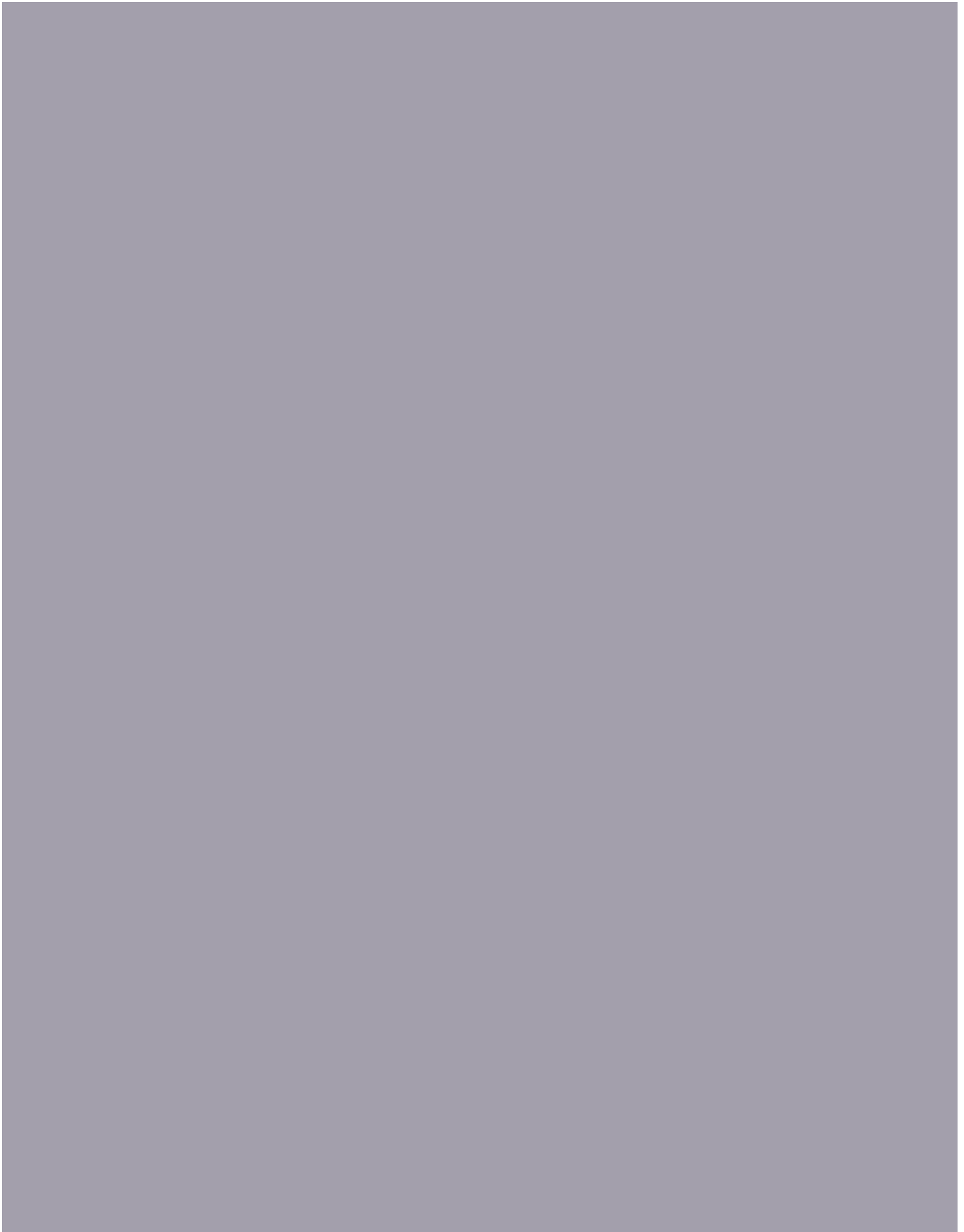
Phase 2 — Customer Management

Weeks 2–3. Goal: Full customer CRUD, offline-first data layer, fuzzy search, and the complete transaction engine with advance wallet logic.











 08 · PHASE 3

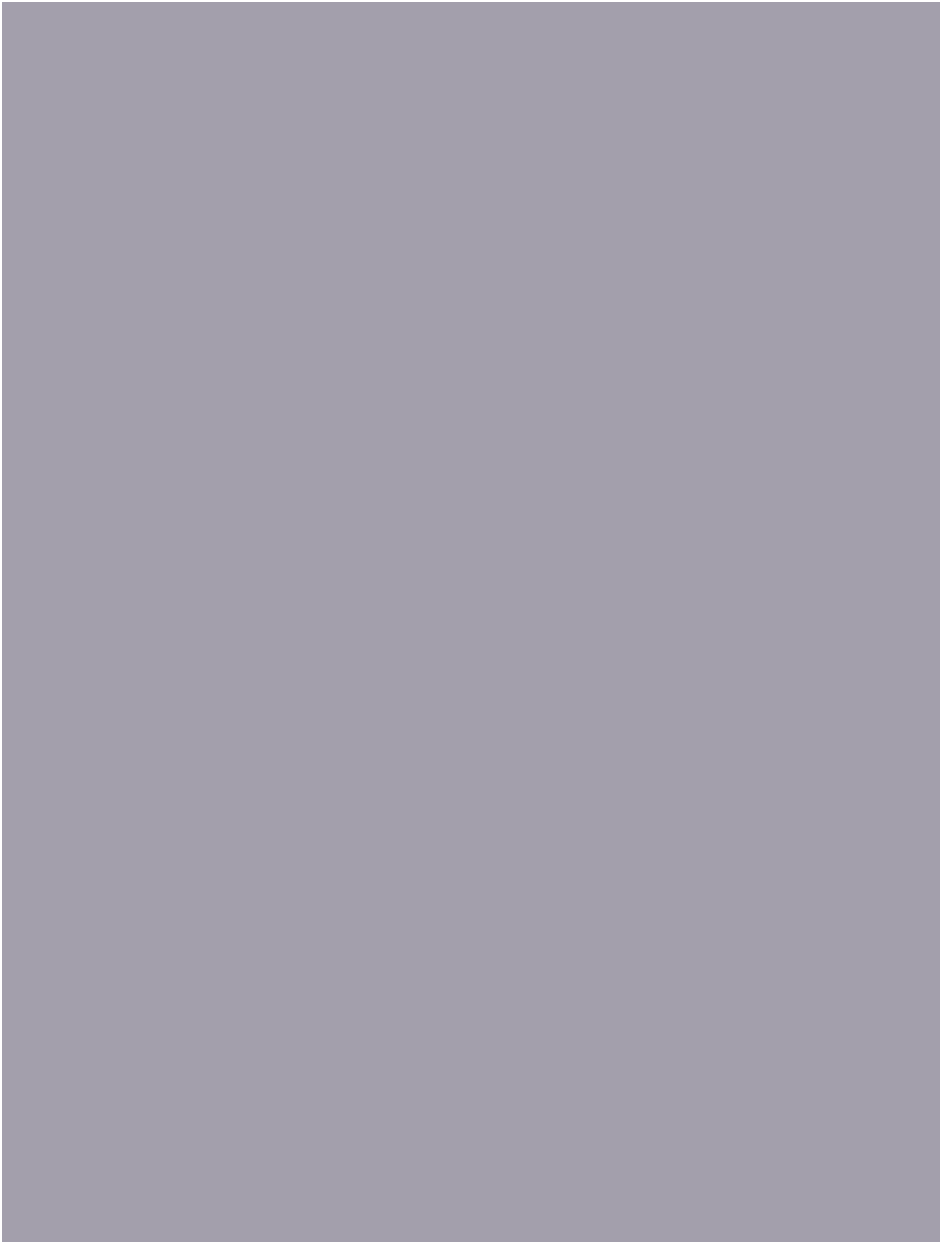
Phase 3 — Reports & Analytics

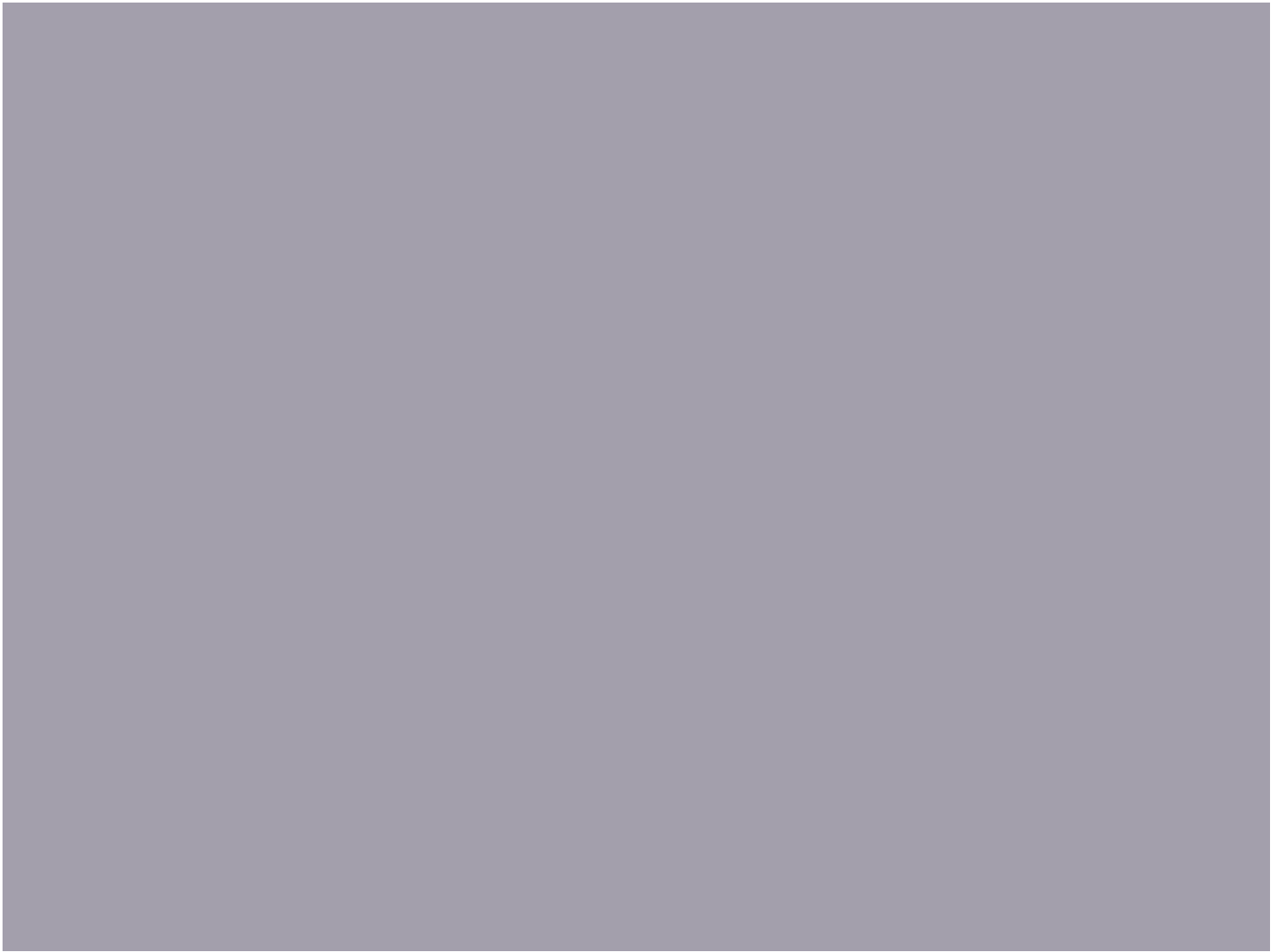
Week 4. Goal: Full business intelligence dashboard with interactive charts, date filtering, and multiple export formats.

— 09 · PHASE 4

Phase 4 — Settings, Security & i18n

Week 5. Goal: Complete settings system, PIN security, full multi-language support, and backup/restore functionality.





— 10 · PHASE 5

Phase 5 — Power Features & Polish

Week 6. Goal: QR cards, dispute workflow, write-offs, family accounts, one-tap repeat, remaining 6 languages, and final UI polish.

11 · DEPLOYMENT

Deployment Plan

Zero-server-management stack. Supabase handles backend, Vercel handles frontend — both scale automatically.

Environment Variables

```
NEXT_PUBLIC_SUPABASE_URL=https://xxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJxxx ...
SUPABASE_SERVICE_ROLE_KEY=eyJxxx ...    # Server-only, never expose to client
NEXT_PUBLIC_APP_URL=https://khataflow.vercel.app
```

12 · TESTING

Verification Plan

Each phase has specific acceptance criteria that must pass before the next phase begins.

Phase	Test Scenario	Expected Result	Priority
Phase 1	Register with email	Verification email received and clickable	Critical
Phase 1	Sign in with Google	OAuth redirect → onboarding → dashboard	Critical
Phase 1	Access protected route logged out	Redirected to /login	Critical
Phase 2	Add customer "Ramesh Kumar"	Appears in list, synced to Supabase DB	Critical
Phase 2	Search "Ramsh" (typo)	Finds "Ramesh Kumar" via fuzzy match	High
Phase 2	Add credit ₹500, advance wallet ₹200	New balance = ₹300, advance = ₹0	Critical
Phase 2	Record payment ₹700 (outstanding ₹500)	Outstanding = ₹0, advance wallet = ₹200	Critical
Phase 2	Send Reminder (WhatsApp)	Correct wa.me URL opens with pre-filled message	High
Phase 2	Refresh browser	All data persists (IndexedDB)	Critical
Phase 2	Go offline, add credit	Data saved locally, syncs when back online	High
Phase 3	Reports page — date range filter	Charts update correctly for selected period	High
Phase 3	CSV export	Downloads valid CSV with correct columns	Medium
Phase 3	Print statement	Opens print dialog with formatted layout	Medium
Phase 4	Switch language to Hindi	All UI strings update immediately	High

PHASE	TEST SCENARIO	EXPECTED RESULT	PRIORITY
Phase 4	Enable PIN lock, reload app	PIN entry required before accessing app	High
Phase 4	Download + re-import backup	All data restored, no duplicates	Medium
Phase 5	Write-off ₹200 (outstanding ₹500)	Outstanding = ₹300, WRITEOFF entry in timeline	High
Phase 5	QR code generation	QR encodes customer URL, scannable	Medium

Build Validation Commands

```
# Run before each phase is marked complete
npm run lint           # Zero ESLint errors
npm run type-check     # Zero TypeScript errors
npm run build          # Production build succeeds
```

SUMMARY

Timeline Summary

Six weeks. Five phases. One production-ready app.



*"Speed, Simplicity, and Reliability — A transaction in under five seconds.
Every feature exists to serve that promise."*

— KhataFlow Core Vision Statement



Web App Implementation Plan v2.0

Next.js 14 · Supabase · Vercel · Prepared July 2026 · Confidential

5 Phases · 6 Weeks · 8 Languages · ∞ Reliability